InsaCloud

Location de conteneurs Linux à la demande, avec accès SSH et haute disponibilité

Mohamed Aziz BAOUEB • Youssef FARJALLAH • Elyes LEMHANNI
STI 4A FISE • Projet 3 • Outils de déploiement de plateformes
Dr. Nadia FETTAH • 2026/2027

Code source : https://github.com/aziz3r/insacloud

1 Introduction

Objectif du projet. InsaCloud est une application web Flask qui permet à un visiteur de créer un compte, puis de *louer* depuis un tableau de bord une machine (conteneur Docker) de la distribution et pour la durée de son choix. Un accès SSH lui est remis au moment de la location, et le service reste disponible même en cas de crash de l'instance ou perte d'accès.

Portée du document. Ce rapport décrit l'architecture retenue, les fonctionnalités livrées et vérifiées, les difficultés rencontrées et les pistes d'évolution. Le projet a été mené par un groupe de trois, là où l'énoncé en prévoit quatre à cinq : les quatre catégories de tâches (API web, partie Ansible, base de données, déploiement des machines) ont donc été réparties entre trois personnes. Un *dossier technique* complémentaire détaille la synthèse de cours et l'analyse de conformité point par point.

Périmètre fonctionnel. Trois distributions (Ubuntu 22.04, Debian 12, Alpine 3.20), deux modes d'accès (terminal ou bureau graphique), durée de 1 à 120 minutes prolongeable, quota de trois machines par utilisateur, destruction automatique à l'expiration.

2 Aperçu de l'architecture

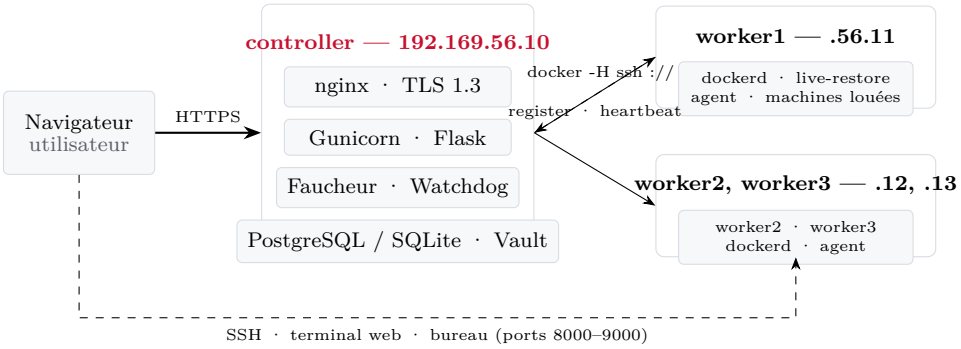


Figure 1 – Architecture déployée : un contrôleur et trois *workers*, provisionnés par Vagrant et Ansible. Chaque worker s'inscrit lui-même auprès du contrôleur et signale sa présence.

Séparation contrôleur / workers. Le serveur web ne doit pas être le point de défaillance unique de l'exécution. Le contrôleur héberge donc uniquement le site, la base et les démons de supervision ; il n'exécute *aucun* conteneur loué et ne possède même pas de démon Docker, mais seulement le client. Les machines louées vivent sur des *workers* dédiés.

Pilotage des workers par SSH. Plutôt que d'exposer l'API Docker sur le réseau (risque d'élévation de privilèges équivalente à un accès `root`), le contrôleur utilise le transport natif du client Docker : `docker -H ssh://insacloud@worker1`. L'authentification se fait par clé dédiée, générée par Ansible et déposée sur les workers, pour un compte sans mot de passe.

Placement des machines. À chaque location, l'application interroge la base pour compter les machines actives par nœud et choisit le moins chargé — un ordonnanceur simple, déterministe et lisible, suffisant pour un parc de cette taille.

Haute disponibilité à deux niveaux. Dans chaque machine, `supervisord` s'exécute en PID 1 et relance individuellement les services (sshd, terminal web, serveur d'affichage, bureau) ; si supervisord meurt, Docker relance le conteneur grâce à `--restart=always`. Le démon Docker des workers est configuré en `live-restore` : une mise à jour de Docker n'interrompt pas les machines en cours.

Nettoyage par un démon dédié (« le Faucheur »). Un service séparé lit la base toutes les dix secondes et détruit les machines expirées. Il réalise aussi une *réconciliation* : un conteneur inconnu de la base est supprimé, une machine sans conteneur est marquée arrêtée, et un worker injoignable est ignoré plutôt qu'interprété comme une disparition.

Auto-réparation du service (watchdog). Un *timer* systemd vérifie chaque minute nginx, la réponse HTTPS du site et le Faucheur, redémarre ce qui est tombé et journalise les workers injoignables.

Infrastructure comme code. Un `Vagrantfile` décrit les quatre machines virtuelles ; un playbook Ansible de neuf rôles les configure intégralement. Le playbook est idempotent : un second passage renvoie `changed=0` sur les quatre nœuds.

Gestion des secrets. Les clés sensibles (session Flask, chiffrement des mots de passe) sont stockées chiffrées dans un fichier Ansible Vault, et déposées sur le serveur dans un fichier en permissions `0600` lu par systemd — jamais dans une unité ni dans les journaux.

3 Fonctionnalités

Comptes et tableau de bord

Inscription et connexion (mots de passe hachés PBKDF2 salé), verrouillage après cinq échecs, redirection vers un tableau de bord qui liste les machines, leur état Docker réel, le temps restant et les actions disponibles.

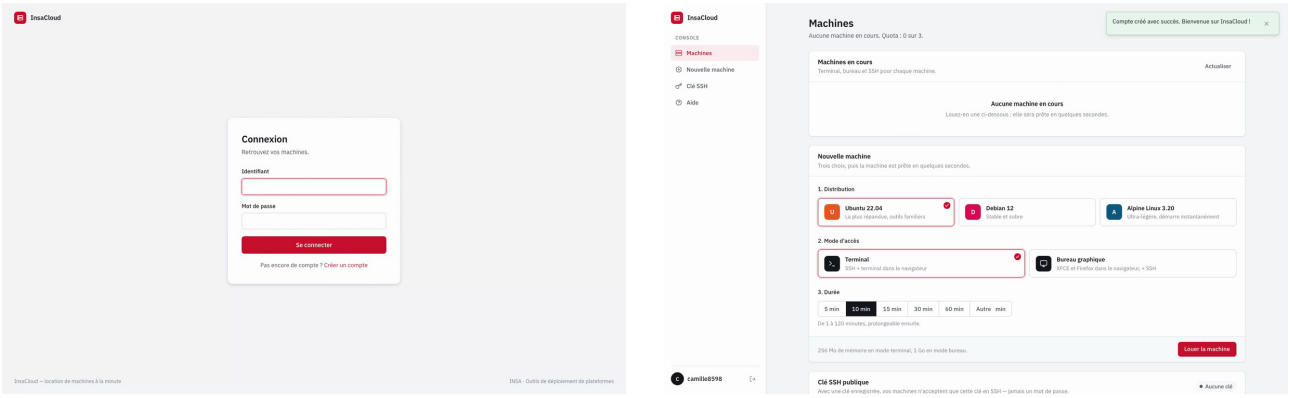


Figure 2 – Connexion (gauche) et tableau de bord avec le formulaire de location en trois choix (droite).

Location : distribution, mode et durée

Trois distributions et deux modes d'accès, soit six images construites à partir de trois `Dockerfile` (la variante bureau s'active par `--build-arg DESKTOP=1`). La commande effectivement exécutée est :

```
docker run -d --restart=always -p <p1>:22 -p <p2>:7681 [-p <p3>:6080] \
  --name insacloud_<user>_<id> -e ROOT_PASSWORD=<aleatoire> [-e SSH_PUBKEY=...] \
  --cap-drop ALL --cap-add CHOWN ... --security-opt no-new-privileges:true \
  --pids-limit 512 --cpus 1 --memory 256m|1g  insacloud_<distro>[_desktop]:latest
```

Accès SSH remis à la location

La commande exacte (`ssh root@<hôte> -p <port>`) est affichée sur la carte de la machine. Le mot de passe `root` est tiré au sort (16 caractères), affiché *une seule fois*, puis conservé chiffré ; il peut être

régénéré à chaud. L'utilisateur peut enregistrer sa clé publique SSH : ses machines n'acceptent alors plus que cette clé.

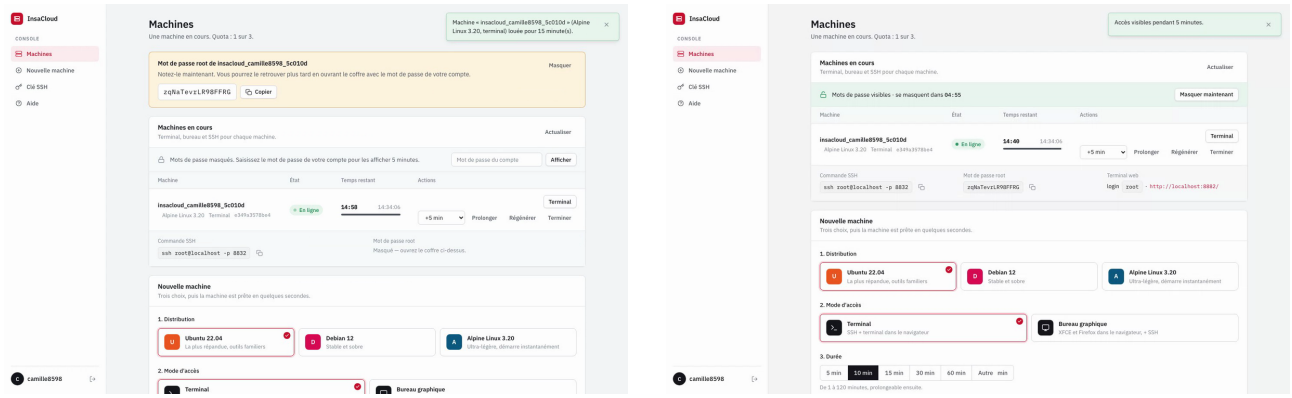


Figure 3 – Mot de passe affiché une seule fois (gauche) ; coffre rouvert après re-authentification (droite).

Accès sans client : terminal et bureau dans le navigateur

Chaque machine expose un terminal web (`ttyd`) avec une invite de connexion classique, et, en mode bureau, un environnement XFCE complet avec Firefox diffusé par noVNC.

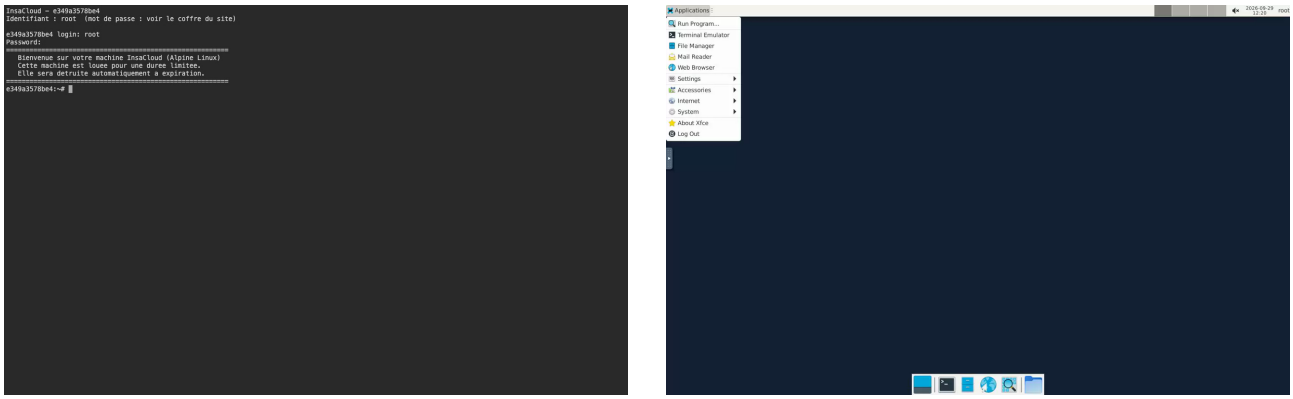


Figure 4 – Terminal web (gauche) et bureau XFCE dans le navigateur (droite).

Un parc qui se gère seul

Aucun nœud n'est déclaré à la main. Un agent installé sur chaque worker l'inscrit au démarrage (`POST /workers/register`) puis signale sa présence toutes les trente secondes. Le contrôleur connaît ainsi en permanence les nœuds joignables, leurs ressources et leur charge, et place chaque nouvelle machine sur le moins chargé. Cet agent n'utilise que la bibliothèque standard : un worker n'a aucun paquet Python à installer.

Haute disponibilité : trois niveaux

Dans la machine, `supervisord` relance un service mort. **Sur le nœud**, `--restart=always` relance le conteneur et `live-restore` lui permet de survivre à un redémarrage du démon Docker. **Dans le parc**, un worker qui cesse de battre passe `OFFLINE` et ses machines sont recréées ailleurs.

Mesuré sur un parc à deux nœuds : nœud éteint, détecté hors ligne puis machine recréée ailleurs **en 31 secondes**, avec un nouveau port et un nouveau mot de passe — mais *la location n'est pas touchée*, l'utilisateur garde la durée qu'il a réservée, et le tableau de bord le lui dit. Le nœud redémarré réintègre le parc en moins de dix secondes.

Précision d'honnêteté : les deux nœuds de cette mesure sont des conteneurs exposant un démon Docker par SSH, et non les machines virtuelles du `Vagrantfile` — aucun hyperviseur ne démarre sur le poste de développement (interfaces réseau VMware absentes, support ARM de VirtualBox défaillant). Le transport SSH, l'inscription des nœuds, le battement de cœur, la bascule `OFFLINE` et la recréation sont ceux du code ; seule la frontière matérielle entre nœuds est simulée. Cette démonstration a d'ailleurs révélé deux défauts réels, corrigés et couverts par des tests : une réconciliation qui détruisait les machines des autres nœuds, et une reprise qui échouait sur une collision de nom.

Ce que la reprise ne rend pas : les fichiers de l'ancienne machine, limite assumée d'une reprise sans stockage partagé.

Expiration

Le Faucheur relit la base toutes les dix secondes. Mesuré : une location d'une minute est détruite 49 s après son échéance, le conteneur retiré du worker et la location passée à `EXPIRED`.

Sauvegarde du travail avant destruction

Détruire un conteneur détruit sa couche inscriptible : l'utilisateur perdrait tout. Juste avant le `docker rm -f`, la plateforme lit un `tar` de `/root` et `/home` au fil de l'eau — sans écrire dans le conteneur — et le conserve sept jours, téléchargeable depuis le tableau de bord avec son empreinte SHA-256. L'archivage ne bloque jamais la destruction : sans cette règle, une machine expirée sur un nœud en panne resterait en vie indéfiniment.

Automatisation complète

`vagrant up` crée les quatre VM et enchaîne lui-même le playbook. Neuf rôles installent pare-feu, Docker, certificats, images, agents, nginx et application. Mesuré sur quatre nœuds : premier passage `failed=0`, second passage `changed=0` partout.

La pile est aussi disponible en conteneurs : un seul `docker-compose.yml` décrit PostgreSQL, l'application, le Faucheur et les images louables. `docker compose up -d` suffit à obtenir une plateforme fonctionnelle, l'application tournant sous un compte sans privilège.

Sécurité

Jetons CSRF sur tous les formulaires, politique de mot de passe, cookies `Secure` / `HttpOnly` / `SameSite=Strict`, en-têtes de sécurité avec une CSP `default-src 'self'` (aucun CDN, aucun script en ligne), HTTPS de bout en bout, pare-feu UFW différencié par groupe, conteneurs à capacités minimales.

Les six tests que le sujet énumère sont scriptés dans `tests/integration.sh` et ont été rejoués sur la plateforme en fonctionnement : **21 vérifications réussies, aucun échec**, une seule ignorée (HSTS, la pile locale étant en HTTP). Ils comprennent une *vraie* connexion SSH à une machine louée et le scénario de panne décrit plus haut.

Ces propriétés ne sont pas seulement décrites : une chaîne d'intégration continue de onze travaux les rejoue à chaque envoi — analyse statique, 169 tests, audit des dépendances, recherche de secrets dans tout l'historique, analyse dynamique de l'application en fonctionnement, démarrage de la pile complète, scan des images, puis publication et déploiement. Aucune image portant une faille grave et corrigeable n'atteint le registre, et chaque image publiée est accompagnée de son inventaire de composants (SBOM).

La chaîne a d'ailleurs trouvé ce que la relecture n'avait pas vu : huit défauts, dont un Faucheur qui détruisait sa propre plateforme et une course entre les processus au démarrage. C'est la raison d'être de l'exercice — un projet qui ne tourne que sur le poste de son auteur n'est pas reproductible, et seule une machine vierge le démontre.

4 Difficultés

La politique `--restart=always` ne se déclenche pas après un `docker kill`. Docker distingue volontairement un arrêt manuel d'un crash. La démonstration de résilience passe donc par la mort du processus *à l'intérieur* du conteneur (`kill -TERM 1`), ce qui relance bien le conteneur (`RestartCount: 1`).

Terminal web inutilisable derrière une authentification HTTP. Protégé par `Basic Auth`, le terminal affichait une page blanche dans plusieurs navigateurs. Le remplacement par `/bin/login` dans le terminal a rendu l'expérience familière et plus sûre (PAM, temporisation entre échecs, expiration de l'invite).

Contraintes des images de distribution. Ubuntu ne fournit Firefox qu'en paquet Snap, inutilisable en conteneur : il a fallu passer par le dépôt APT officiel de Mozilla. `ttyd` n'est pas empaqueté dans Debian 12 : binaire statique officiel. Enfin, le nom du binaire de mot de passe VNC diffère selon la distribution, d'où une détection à l'exécution.

Un mot de passe VNC est limité à huit caractères par le protocole : il a fallu exposer un mot de passe de bureau distinct et chiffrer le transport.

Docker dans Docker pour tester le cluster. Le banc de test (trois conteneurs systemd simulant les VM) échouait au `docker build` : le pilote overlay ne peut pas se superposer à lui-même. La solution a été de monter `/var/lib/docker` et `/var/lib/containerd` sur des volumes Docker dédiés.

Deux tâches Ansible en conflit sur le même répertoire. Le rôle `controller_key` créait `.ssh` en `0700` et le rôle `webapp` le remettait en `0755` : chaque exécution affichait un `changed`. Le retrait du doublon a rendu le playbook parfaitement idempotent.

PEP 668. Depuis Ubuntu 23.04, `pip install` est interdit hors environnement virtuel, ce qui impose de créer un *virtualenv* dans le rôle Ansible.

5 Orientations futures

Déjà comblé depuis la première version. La relecture du cours avait révélé huit écarts, tous traités depuis : un fichier `docker-compose.yml` déclare la construction des images et une machine de démonstration ; chaque conteneur reçoit désormais une *réservation* mémoire en plus de sa limite ; la construction des images est encadrée par `block`/`rescue`/`always` ; le `Vagrantfile` enchaîne lui-même le *provisioner* Ansible ; le *watchdog* relève la consommation CPU et mémoire de chaque machine (`docker stats`) ; les neuf rôles déclarent leurs dépendances dans `meta/main.yml` ; un inventaire dynamique interroge Vagrant ; enfin une chaîne d'intégration continue exécute à chaque envoi onze travaux — lint, 169 tests, SAST, SCA, recherche de secrets, DAST et scan de l'image Docker.

Court terme. Certificat reconnu (Let's Encrypt) en remplacement du certificat auto-signé, seule alerte que le scan dynamique signale encore ; quotas de ressources par utilisateur et non plus seulement par machine ; notification (courriel ou webhook) lorsqu'un worker devient injoignable.

Moyen terme. Authentification à deux facteurs ; journalisation centralisée des accès aux machines louées ; archivage du travail de l'utilisateur avant destruction, pour que l'expiration ne soit plus une perte sèche.

Long terme. Passage à un orchestrateur (Docker Swarm ou Kubernetes) si le parc dépasse la dizaine de nœuds : la planification, la santé des nœuds et le redémarrage des conteneurs seraient alors délégués à l'orchestrateur, et le Faucheur se réduirait à une politique d'expiration.